Project Sentinel — ET AI Hackathon 2.0 Build Documentation

Problem Statement: PS2 — Agentic AI for Autonomous Enterprise Workflows **Vertical:** Autonomous Security Operations (SOC) Incident Response **Stack:** MERN + Node orchestration FSM + Gemini/Claude API + threat-intel APIs

---

1. What This Is

Sentinel is a five-agent autonomous system that takes a raw security alert, investigates it, decides what to do about it, and proves — cryptographically and visibly — exactly how it made that decision. It's built for security operations teams who are drowning in alert volume and can't afford an analyst spending 15–30 minutes manually triaging every ping.

The pitch in one line: **"A SOC analyst that never sleeps, never gets fatigued, and shows its work on every single decision."**

2. The Problem & Why It Matters

Enterprise SOC teams face an alert-volume problem, not a detection problem — most tools are good at flagging anomalies, few are good at deciding what to do about them fast enough to matter. The cost isn't hypothetical: mean-time-to-respond directly correlates with breach cost and blast radius. And every automated remediation system enterprises actually adopt has to clear a trust bar — "why did the system do that, and can I audit it" — or compliance and security leadership won't sign off on giving it real autonomy. Sentinel is designed around that trust bar as the core product, not an afterthought.

3. Architecture (Recap)

Five agents, sequential handoff, one conditional retry loop: **Detection → Enrichment → Triage (confidence-gated, retries once, else escalates) → Decision (constrained action space) → Audit (writes to hash-chained ledger)**

Full architecture detail is in the prior strategy doc — this document focuses on what's *new*: the two soft spots we're now hardening.

4. Strengthened Differentiators

4A. Innovation & Creativity — upgraded

The original hash-chained audit ledger was good but passive. Three additions make it actively demonstrable, not just displayed:

1. **"Break the Chain" judge-interactive panel.** Judges can click any past ledger entry, edit a single field (e.g. change a logged confidence score), and hit re-verify. The UI visibly shows the chain breaking from that point forward — every subsequent hash turns red. This turns "immutable audit trail" from a claim into something a skeptical judge can personally falsify-test in fifteen seconds. This is the single highest-leverage addition in this whole update.
2. **Counterfactual reasoning display.** When the Triage agent classifies an incident, it also logs the *second-most-likely* classification and why it was rejected (e.g. "83% DDoS, 14% false-positive — rejected because request pattern doesn't match known scanning tools"). This directly answers the "is this just a black box" objection before a judge can raise it, and it's a genuinely uncommon feature — most hackathon agent demos show only the winning decision.
3. **Live confidence-calibration view.** A small running chart plotting the Triage agent's stated confidence against a synthetic "ground truth" label for each seeded case. This demonstrates the system knows when it doesn't know — the actual technical substance behind "self-correcting," rendered as evidence instead of asserted as a bullet point.

4B. Presentation & Clarity — upgraded

1. **Judge-injected alert mode.** Instead of only running a pre-scripted incident, give the jury a button: "Inject your own alert" — a small form (source IP, request pattern, target) they fill in live. The pipeline runs on their input in real time. This is the single best way to prove the demo isn't a recorded illusion, and it makes the jury active participants instead of passive viewers — which measurably increases how memorable a pitch is in a back-to-back demo day.
2. **Split-screen cold open.** First 15 seconds: left side, a stopwatch counting up next to "Analyst manually triaging..." with a static, boring screen; right side, Sentinel's agent graph completing the same case in real time. Stop the clock at the moment Sentinel finishes. This single visual does more work than any slide of bullet points for the "impact" story.
3. **One-sentence rule for every panel.** Each UI panel gets a single caption sentence visible at all times (e.g. under the ledger: "Every decision is logged and tamper-

evident — click any entry to verify"). Judges rotate through many demos; a panel that explains itself without narration survives the moments when your presenter is mid-sentence answering a question instead of pointing at the screen.

---

5. Updated Judging Criteria Alignment

Criterion	Coverage
Relevance	Direct 1:1 with PS2's "owns a process, self-corrects, auditable trail"
Innovation	Break-the-Chain + counterfactual display — both novel, both demoable in seconds
Technical Implementation	Real threat-intel API calls, real hash-chain verification, real confidence-gated retry logic
Business Viability	MTTR reduction, analyst-hours saved, SOC2/ISO27001 audit-readiness angle
Presentation & Clarity	Judge-injected live mode + split-screen cold open removes all "is this scripted" doubt
Impact & Scalability	Same 5-agent pattern reusable for cost leakage / compliance verticals

---

6. The Antigravity Build Prompt

Antigravity works best from one detailed upfront spec rather than many small back-and-forth prompts, since it plans the whole task graph before dispatching sub-agents to write, test, and verify code. Paste the block below as your first prompt in a new Antigravity workspace. Set the mode to **Agent-assisted** (not full Autopilot) for this build — you want to review the orchestration/FSM logic and the ledger hash-chain code yourself before it's marked done, since those two pieces are your core differentiators and the ones you'll be questioned on by judges.

PROJECT: Sentinel — Autonomous SOC Incident Response System (hackathon prototype)

CONTEXT
This is a hackathon prototype (not production software) demonstrating a five-agent autonomous pipeline that ingests security alerts, investigates them, decides on remediation, and writes every decision to a tamper-evident audit ledger. The judged deliverable is a working, demoable prototype — prioritize a reliable end-to-end flow over exhaustive edge-case handling.

STACK
- Frontend: React (Vite), Tailwind CSS, Framer Motion for animations, Socket.IO client
- Backend: Node.js + Express, Socket.IO server, MongoDB (Mongoose)
- Orchestration: a hand-rolled finite-state machine in Node — do NOT use a heavyweight agent framework (no LangGraph/CrewAI/AutoGen). Each agent is a plain async function taking and returning a shared `caseState` object.
- LLM calls: abstract behind a single `callLLM(systemPrompt, userPayload, jsonSchema)` function so the model provider can be swapped. Use structured JSON output (function-calling / schema-constrained) for every agent call — never free-text parsing.
- External APIs: AbuseIPDB free tier, VirusTotal public API, GreyNoise Community API. Wrap each in a small client module with graceful fallback to cached/mock responses if a key is missing or the call fails, so the demo never breaks on network flakiness.

DATA MODEL (MongoDB)
- `alerts`: raw incoming alert records (source IP, timestamp, pattern type, raw payload)
- `cases`: one per alert being processed, holds current FSM state, all agent outputs, final confidence and classification
- `ledger`: append-only collection. Each document: { caseId, agentName, timestamp, inputHash, outputHash, prevHash, action, confidenceScore }. `inputHash`/`outputHash` = SHA-256 of the JSON-stringified input/output. `prevHash` = hash of the previous ledger entry for that case (chain). Include a `verifyChain(caseId)` function that recomputes all hashes in sequence and returns which entry (if any) breaks the chain — this powers the "Break the Chain" judge-interactive feature, so build it as a real, callable verification function, not a cosmetic check.

THE FIVE AGENTS (implement as a sequential FSM with one conditional loop)

1. DetectionAgent — deterministic, NOT an LLM call. Takes a raw alert, applies simple threshold rules (request-rate spike, presence on a known-bad-IP list) to decide if it's worth investigating. Writes ledger entry on completion.

2. EnrichmentAgent — calls AbuseIPDB + VirusTotal + GreyNoise in parallel (Promise.all), then one LLM call to summarize the combined results into a structured `ThreatContext` object: { severity, confidenceHint, iocList, summary }. Writes ledger entry.

3. TriageAgent — LLM call. Takes ThreatContext + case history, returns a structured classification: { primaryClassification, primaryConfidence, secondaryClassification, secondaryConfidence, rejectionReason }. secondaryClassification/rejectionReason are REQUIRED fields in the schema — this is the counterfactual-reasoning feature, do not make it optional or the frontend feature breaks.
   SELF-CORRECTION RULE: if primaryConfidence < 0.7, call EnrichmentAgent again with a narrower follow-up question appended to context, then re-run TriageAgent once. If still < 0.7 after retry, set case status to "escalated_to_human" and stop the pipeline — do not force a low-confidence auto-decision. Log both the retry attempt and the escalation as separate ledger entries so the retry is visible in the UI, not hidden.

4. DecisionAgent — LLM call constrained to a fixed action enum: ["rate_limit_ip", "quarantine_host", "open_ticket", "notify_oncall", "no_action_false_positive"]. Never allow freeform action text. Writes ledger entry with the chosen action and the confidence it acted on.

5. AuditAgent — not an LLM call. Purely appends the final case summary to the ledger and marks the case resolved. Emits a Socket.IO event so the frontend ledger panel updates live.

FRONTEND — three-panel dark glassmorphism dashboard
- Background near-black (#0a0a0f), thin 1px low-opacity-white borders, backdrop-filter blur on all panels, monospace font for IOC/ledger data.
- LEFT PANEL: live alert feed, severity-color-coded cards.
- CENTER PANEL: animated agent graph (5 nodes in a line: Detection → Enrichment → Triage → Decision → Audit). Active agent glows/pulses. During a Triage retry, render a visible loop-back edge from Triage to Enrichment in amber before it re-runs. Include a persistent one-sentence caption under this panel explaining what it shows.
- RIGHT PANEL: scrolling ledger list, each entry expandable to show agentName, timestamps, input/output hashes, and confidence. Include a "Verify Chain" button that calls verifyChain(caseId) and highlights the

full chain green if valid. Include an editable-entry demo mode: an "edit entry" pencil icon on each ledger row that lets a user change the stored confidenceScore value and re-run verifyChain — every entry from that point forward should visibly turn red, demonstrating the tamper-evidence live. This is a REQUIRED feature, not a stretch goal — it is the core "Innovation" differentiator for this prototype.
- TOP BAR: a small form labeled "Inject Custom Alert" (source IP, pattern type dropdown, target field) that lets a user submit a live alert through the real pipeline instead of only the seeded demo cases. This is also a REQUIRED feature — it proves the demo isn't scripted.
- A confidence-calibration mini-chart (simple line/scatter, can use Recharts) plotting TriageAgent confidence vs a stored "expectedOutcome" field on each seeded case, so the system's calibration is visible, not just claimed.

SEED DATA
Create 6–8 pre-built alert cases covering a mix of clear DDoS, clear false-positive, and 1–2 deliberately ambiguous cases designed to trigger the confidence retry/escalation path at least once — the retry path must be reliably demoable, don't leave it to chance.

WHAT TO MOCK (explicitly, and note it in code comments as "MOCKED FOR DEMO")
- Actual firewall/network remediation calls — log and simulate the action, do not attempt real network enforcement.
- Historical case similarity search — a small in-memory/seeded lookup is fine, no need for a real vector search index.

BUILD ORDER (please plan and confirm this sequence before writing code)
1. MongoDB schemas + hash-chain ledger module + verifyChain function (with unit tests proving tamper detection actually works)
2. Five agent functions wired into the FSM, testable via a CLI/script before any UI exists
3. Express API + Socket.IO event wiring
4. React dashboard: agent graph panel first, then ledger panel with verify/tamper feature, then alert feed + inject-alert form, then calibration chart
5. Seed script for the 6–8 demo cases
6. End-to-end smoke test: run all seeded cases through the full pipeline and confirm ledger integrity holds for all of them

Please start by proposing the file/folder structure and the ledger schema, and confirm both with me before generating the rest of the codebase.

- Set Antigravity's mode to **Agent-assisted**, not Autopilot, for this build — you want the chance to review the FSM and ledger logic before it's marked complete.
- If Antigravity supports custom **rules**, add one pinning: no hardcoded API keys, use `.env`, and require the LLM calls to use schema-constrained JSON output — this keeps every agent's output parseable and avoids the classic "LLM returned prose instead of JSON" demo-breaking bug.
- Review the `verifyChain` function yourself line by line once it's generated — this is the piece a judge is most likely to ask you to explain, so you need to actually understand it, not just trust that it was generated correctly.

---

7. Demo Script (tightened for the new features)

1. **0:00–0:15** — Split-screen cold open: manual analyst stopwatch vs. Sentinel's agent graph, same case, side by side.
2. **0:15–0:45** — Narrate the five-agent graph lighting up node by node on a seeded DDoS case; call out the retry loop explicitly if it fires ("notice it didn't guess — it went back for more context").
3. **0:45–1:05 — Judge-injected alert**: hand the mic/keyboard moment to a judge, let them submit a made-up alert, show it flow through live.
4. **1:05–1:30 — Break the Chain**: edit a past ledger entry live, show the chain turn red from that point forward, then explain why this matters for compliance/audit adoption.
5. **1:30–1:50** — Impact numbers on screen (MTTR reduction, analyst-hours saved, audit-readiness).
6. **1:50–2:00** — Close on scalability: same pattern, different action space, works for cost leakage or compliance verticals too.

---

8. Pre-Submission Checklist

- [ ] All five agents run end-to-end on all seeded cases without manual intervention
- [ ] At least one seeded case reliably triggers the confidence retry/escalation path
- [ ] `verifyChain` correctly detects tampering on a manually edited entry (test this yourself before the demo, don't assume)
- [ ] Judge-inject-alert form works against the real pipeline, not a mocked response
- [ ] No hardcoded API keys anywhere in the repo (check before pushing to a public GitHub link)

- [ ] README clearly states what's real vs. simulated (per your hackathon's originality/plagiarism rules — be explicit and honest about mocked pieces)
- [ ] Demo video ⩽ 2 minutes, rehearsed at least twice against a clock